



ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR

Membre de Honoris United Universities

Filiere Ingenierie Informatique et Reseaux

PROJET DE FIN D'ANNEE

3^{eme} Annee en Ingenierie Informatique et Reseaux

LogAnalyze

*Plateforme centralisee de collecte, d'audit de securite
et d'analyse analytique des journaux Nginx multi-serveurs*

Realise par :

Elias Chakroun
Yahya Khalifi

Tuteurs :

Encadrant Pedagogique :
Pr. Sara AIT BENNACER
Encadrant Professionnel :
Madame ER-RAGHI Karima

Organisme d'accueil : SOPHATEL

Annee universitaire : 2025 — 2026

Dedicaces

Nous dedions ce modeste travail

*A nos chers parents,
pour leur soutien indefectible, leurs sacrifices constants
et leurs prieres qui eclairent notre chemin ;*

*A nos freres et soeurs, nos familles et proches,
pour leurs encouragements chaleureux tout au long de notre cursus ;*

*A tous nos professeurs de l'EMSI,
pour la rigueur et la qualite des enseignements dispenses ;*

*A l'equipe d'ingenierie de SOPHATEL,
pour leur confiance, leur disponibilite et leur encadrement methodologique.*

Remerciements

Au terme de ce projet de fin d'année, nous tenons à témoigner notre profonde reconnaissance à l'ensemble des acteurs ayant contribué à l'aboutissement de nos travaux.

Nous exprimons nos sincères remerciements à notre encadrante pédagogique à l'**Ecole Marocaine des Sciences de l'Ingenieur (EMSI)**, le Professeur Sara AIT BENNACER, pour son suivi méthodologique, ses conseils avisés et ses exigences académiques.

Nous adressons également notre gratitude à l'équipe technique de la société **SOPHATEL** pour l'accueil accordé durant ce stage. Leur mentorat sur les architectures distribuées, l'administration de serveurs de production et l'analyse de logs à grand volume a été déterminant pour concilier théorie logicielle et réalités de terrain.

Enfin, nos remerciements s'adressent au corps professoral de l'EMSI pour la formation d'ingénierie dispensée, ainsi qu'à tous nos proches pour leur bienveillance permanente.

Resume

Le present rapport expose les travaux conduits dans le cadre du projet de fin d'annee realise au sein de la societe **SOPHATEL**. Le projet, intitule **LogAnalyze**, porte sur l'ingenierie, la reprise et l'industrialisation d'une plateforme distribuee destinee a la collecte, au traitement analytique et a l'observation securitaire des journaux d'accès (*access logs*) Nginx issus d'un parc heterogene de serveurs virtuels prives (VPS).

Loin d'un developpement initialise *ex nihilo*, notre mission s'est focalisee sur une problematique courante du milieu industriel : l'audit approfondi, la fiabilisation et le refactoring d'un socle applicatif preexistant, lequel presentait des dysfonctionnements critiques (defaillances du pipeline de collecte, duplications massives, absence de controle d'accès granulaire et instabilites front-back). Notre intervention a permis de restructurer le backend sous FastAPI, de formaliser le moteur de collecte securise SSH/Paramiko, d'integrer un pipeline de deduplication robuste sous PostgreSQL avec insertion en masse, et de developper une interface reactive sous Angular 17. L'application a ensuite ete enrichie par la geolocalisation d'IPs et conteneurisee integralement via Docker Compose.

Mots-cles : LogAnalyze, Nginx, FastAPI, Angular, PostgreSQL, Paramiko, SSH, Deduplication, Docker, RBAC, PFA, EMSI.

Table des matieres

Dedicaces	i
Remerciements	ii
Resume	iii
Liste des figures	vii
Liste des tableaux	viii
Liste des abreviations	ix
Introduction generale	1
1 Presentation du cadre de projet	2
1.1 Introduction	2
1.2 Presentation de la societe d'accueil	2
1.3 Etude de l'existant	3
1.3.1 Description de la solution heritee	3
1.3.2 Critique de l'existant	3
1.3.3 Solution proposee	3
1.4 Choix du modele de developpement	4
1.5 Planning previsionnel du stage	4
1.6 Conclusion	4
2 Specification des besoins	5
2.1 Introduction	5
2.2 Identification des acteurs	5
2.3 Specification des besoins fonctionnels	5

2.3.1	Besoin fonctionnel 1 : Authentification et controle d'accès	5
2.3.2	Besoin fonctionnel 2 : Gestion du parc de serveurs VPS	6
2.3.3	Besoin fonctionnel 3 : Collecte des journaux d'infrastructure	6
2.3.4	Besoin fonctionnel 4 : Analyse, parsing et deduplication	6
2.3.5	Besoin fonctionnel 5 : Restitution visuelle et alertage	6
2.4	Specification des besoins non fonctionnels	6
2.5	Presentation des cas d'utilisation	7
2.5.1	Diagramme global des cas d'utilisation	7
2.5.2	Description textuelle formalisee des cas d'utilisation	8
2.6	Conclusion	10
3	Conception du systeme	11
3.1	Introduction	11
3.2	Modelisation dynamique	11
3.2.1	Diagramme de sequence	11
3.2.2	Diagramme d'activite	12
3.2.3	Diagramme d'etats-transitions	14
3.3	Modelisation statique	14
3.3.1	Diagramme de classes	14
3.3.2	Modele relationnel	15
3.3.3	Dictionnaire de donnees	16
3.4	Architecture de deploiement de l'application	17
3.5	Conclusion	18
4	Realisation et mise en oeuvre	19
4.1	Introduction	19
4.2	Environnement de developpement	19
4.2.1	Environnement materiel	19
4.2.2	Environnement logiciel	19
4.3	Ameliorations techniques fondamentales apportees a l'existant	20

4.3.1	Ingestion par lots et fiabilisation de la deduplication	20
4.3.2	Centralisation des exceptions et assainissement des reponses	21
4.4	Principales interfaces graphiques	21
4.5	Campagne de tests et validation empirique	23
4.6	Audit de securite de la plateforme	24
4.7	Deploiement conteneurise	24
4.8	Conclusion	24
Conclusion generale et perspectives		25
Bibliographie et netographie		26
A Annexes techniques		27
Annexes		27

Table des figures

1.1	Chronogramme d'avancement des taches du projet	4
2.1	Diagramme des cas d'utilisation de LogAnalyze	8
3.1	Diagramme de sequence : Collecte et analyse des logs	12
3.2	Diagramme d'activite du pipeline de bout en bout	13
3.3	Automate a etats-transitions du moteur d'analyse	14
3.4	Diagramme de classes de l'application LogAnalyze	15
3.5	Diagramme de deploiement physique de l'architecture conteneurisee	18
4.1	Mire de connexion securisee et emission du jeton JWT	21
4.2	Tableau de bord operationnel des indicateurs cles	22
4.3	Console de gestion du parc de serveurs VPS surveilles	22
4.4	Explorateur pagine et filtre des evenements d'accès	23

Liste des tableaux

1.1	Fiche signalétique de la société SOPHATEL	2
1.2	Planning prévisionnel d'exécution des tâches	4
2.1	Synthèse des exigences non fonctionnelles	7
2.2	Description du cas d'utilisation : Collecter les logs d'un VPS	9
2.3	Description du cas d'utilisation : Analyser et dédupliquer les logs	10
3.1	Dictionnaire de données — Table <code>vps_servers</code>	16
3.2	Dictionnaire de données — Table <code>log_entries</code>	17
4.1	Spécifications matérielles du poste de travail	19
4.2	Outillage logiciel et environnements d'exécution	20
4.3	Recueil des scénarios de validation fonctionnelle automatisée	23
4.4	Synthèse de l'audit de sécurité opérationnel	24

Liste des abreviations

Abreviation	Signification
API	Application Programming Interface
CLF	Common Log Format (Format standard Nginx/Apache)
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
EMSI	Ecole Marocaine des Sciences de l'Ingenieur
HTTP	HyperText Transfer Protocol
IP	Internet Protocol
JWT	JSON Web Token
KPI	Key Performance Indicator
ORM	Object-Relational Mapping
PFA	Projet de Fin d'Annee
RBAC	Role-Based Access Control
REST	Representational State Transfer
SFTP	SSH File Transfer Protocol
SSH	Secure Shell
UML	Unified Modeling Language
VPS	Virtual Private Server

Introduction generale

A l'ere des architectures logicielles micro-services et des infrastructures cloud distribuees, les serveurs web hebergeant des applications a fort trafic generent continuellement des volumes considerables de journaux d'evenements (*logs*). Ces journaux constituent la trace operationnelle fondamentale pour le diagnostic des pannes, la mesure des performances et l'analyse forensique post-incident. Neanmoins, lorsque l'infrastructure de production se compose de multiples serveurs distants, l'inspection manuelle des fichiers s'avere inefficace, chronophage et sujette a des erreurs d'interpretation humaine.

C'est pour repondre a cette exigence de centralisation et de supervision proactive que la societe **SOPHATEL** a engage le developpement du systeme **LogAnalyze**. Cette plateforme a pour mandat d'orchestrer la collecte automatique, securisee et periodique des journaux d'accès de serveurs virtuels prives (VPS), d'en operer la normalisation, et de synthetiser les flux sous la forme d'un tableau de bord analytique dote de metriques de performance cles (latences p95/p99, distribution des erreurs 4xx/5xx, routes critiques).

Dans le cadre de notre projet de fin d'annee, nous avons pris la responsabilite de ce projet a partir d'un socle preliminaire. Notre mission technique n'etait pas de developper un prototype a partir d'une page vierge, mais d'exercer un role complet de reprise de projet : auditer l'existant, resoudre les goulets d'etranglement fonctionnels et securitaires, integrer des mecanismes d'ingestion massive dedupliques, et fournir une application documentee, testee et prete a l'exploitation.

Le present memoire rend compte de l'integralite de notre demarche methodologique et technique :

- Le **Chapitre 1** pose le cadre general du projet, introduit l'organisme d'accueil, analyse de maniere critique l'existant et formalise le planning d'execution.
- Le **Chapitre 2** detaille la specification formelle des besoins fonctionnels et non fonctionnels, soutenue par les cas d'utilisation et la description normalisee des scenarios.
- Le **Chapitre 3** expose la demarche de conception statique et dynamique a l'aide du formalisme UML et presente le modele relationnel associe.
- Le **Chapitre 4** rend compte de la realisation effective du systeme, de l'environnement technologique, des ameliorations architecturales majeures, de la validation par les tests et de la conteneurisation.

Chapitre 1

Presentation du cadre de projet

1.1 Introduction

Ce premier chapitre etablit le perimetre institutionnel et organisationnel au sein duquel s'est deroule notre stage. Nous y introduisons la societe SOPHATEL, realisons une etude analytique et critique du socle preexistant, explicitions la problematique d'ingenierie resolue, et precisons la methodologie de gestion de projet retenue.

1.2 Presentation de la societe d'accueil

Fondee en **1983**, la societe **SOPHATEL** est une entreprise marocaine pionniere dans l'edition logicielle et les services numeriques specialises a destination des officines pharmaceutiques et des professionnels de la sante.

TABLE 1.1 – Fiche signalétique de la societe SOPHATEL

Critere	Detail de l'organisme
Raison sociale	SOPHATEL S.A.
Date de fondation	1983
Domaine d'activite	Ingenierie logicielle pour la sante et le secteur officinal
Produits cles	Systemes de gestion d'officine (ERP officinal), EDI pharmaceutique
Valeurs fondamentales	Robustesse, confidentialite des donnees de sante, continuite de service
Implantation	Casablanca, Maroc

Les produits deployes par SOPHATEL traitent quotidiennement des flux critiques (ordonnances, approvisionnements, transactions financieres). La disponibilite et les temps de reponse des serveurs web sous-jacents conditionnent directement l'activite des utilisateurs. La maitrise des journaux d'infrastructure constitue donc un axe operationnel strategique.

1.3 Etude de l'existant

1.3.1 Description de la solution heritee

Au debut de nos travaux, une ebauche preliminaire de la solution LogAnalyze etait operationnelle en environnement restreint. Ce prototype comportait :

- Un script de collecte base sur Paramiko telechargeant des logs bruts,
- Une base de donnees schema basique sous PostgreSQL,
- Un debut de backend FastAPI exposant des routes d'extraction,
- Une interface Angular esquissant la visualisation de metriques de base.

1.3.2 Critique de l'existant

Une campagne d'audit technique et architectural a mis en lumiere des faiblesses substantielles entravant une exploitation en production :

1. **Instabilite du pipeline de collecte** : Des ruptures de flux SSH intervenaient sans reprise sur erreur, creant des enregistrements de collections orphelines et vides dans la base de donnees.
2. **Absence de mecanisme de deduplication** : La collecte repetee du meme journal distraait provoquait l'insertion multiple d'evenements identiques, faussant l'integralite des indicateurs volumetriques.
3. **Lacunes de securite graves** : Aucun controle d'accès base sur les roles (RBAC) n'etait implemente ; un utilisateur authentifie avec des privileges restreints pouvait lancer des purges ou modifier les identifiants serveurs. De plus, les cles privees SSH etaient stockees en clair.
4. **Inefficience algorithmique** : L'analyse des fichiers CSV etait effectuee ligne par ligne via des insertions SQL unitaires, entrainant un cout temporel prohibitif des que le fichier depassait 10 000 lignes.
5. **Absence de pagination** : Les requetes API de consultation renvoyaient l'integralite des enregistrements d'une table, generant des saturations memoire cote frontend.

1.3.3 Solution proposee

Pour remedier a ces insuffisances, nous avons preconise une demarche de refonte structuree :

- Refactoring integral de la couche d'ingestion avec prise en charge du *bulk insert* via SQLAlchemy et verification d'unicite par empreinte,
- Cryptage symetrique (Fernet) des secrets de connexion distants et verification granulaire des roles administratifs,
- Normalisation des statuts de traitement (PENDING, POPULATING, COMPLETE, FAILED),
- Industrialisation du deploiement via des images Docker multi-stages.

1.4 Choix du modele de developpement

Afin de concilier la contrainte d’un delai d’execution resserre et la necessite d’effectuer des retours reguliers aupres de l’equipe technique, nous avons adopte une approche agile adaptee de la methodologie **Scrum**. Cette conduite par iterations d’une semaine a favorise des cycles rapides d’audit, de correction, d’enrichissement fonctionnel et de validation.

1.5 Planning previsionnel du stage

Le calendrier de developpement et de fiabilisation a ete structure sur une periode de quatre semaines intensives.

TABLE 1.2 – Planning previsionnel d’execution des taches

Semaine	Phase methodologique	Livrables & Objectifs
S1	Audit et cadrage	Cartographie des bugs, audit de securite, environnement de travail normalise.
S2	Fiabilisation du pipeline	Correction Paramiko, moteur de deduplication, bulk insert, pagination des routes API.
S3	Securite	Chiffrement des identifiants, controle d’acces RBAC, enrichissement geographique, gestion centralisee des exceptions.
S4	Finalisation et tests	Validation unitaire et fonctionnelle, conteneurisation Docker, documentation et redaction du rapport.

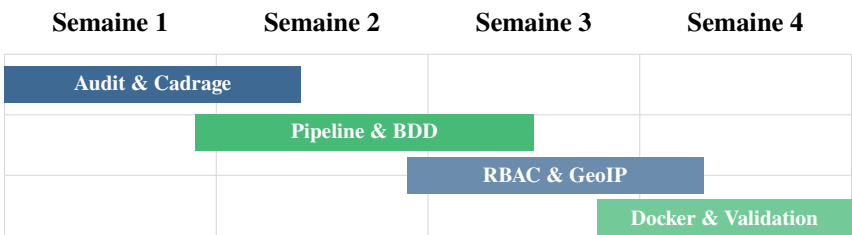


FIGURE 1.1 – Chronogramme d’avancement des taches du projet

1.6 Conclusion

La formalisation du cadre de projet a permis de transformer un code experimental fragile en un projet d’ingenierie cible. Fort de ces bases d’audit, le chapitre suivant precise l’inventaire rigoureux des exigences fonctionnelles et metier.

Chapitre 2

Specification des besoins

2.1 Introduction

L'analyse des besoins constitue le pivot entre la volonté métier de fiabilisation et les arbitrages d'architecture. Ce chapitre structure l'expression des exigences fonctionnelles et qualitatives, explicite les acteurs du système et fournit les descriptions textuelles formalisées des cas d'utilisation selon les standards d'ingénierie logicielle de l'EMSI.

2.2 Identification des acteurs

Dans le cadre de l'exploitation de LogAnalyze, l'interaction avec le système est concentrée et sécurisée :

- **Super-administrateur** : Profil à hauts privilèges chargé de l'administration intégrale de la plateforme. Il configure les inventaires de serveurs VPS, configure les clés d'accès, déclenche ou planifie les cycles de collecte, supervise les rapports de performances et gère la politique de sécurité.

2.3 Specification des besoins fonctionnels

Les besoins fonctionnels expriment les capacités directes développées et fiabilisées dans le système :

2.3.1 Besoin fonctionnel 1 : Authentification et contrôle d'accès

- **Authentification forte** : Délivrance d'un jeton cryptographique JWT après validation du condensat de mot de passe via l'algorithme robuste Argon2id.
- **Gestion de profil** : Vérification des autorisations administratives précédant chaque exécution d'un endpoint critique.

2.3.2 Besoin fonctionnel 2 : Gestion du parc de serveurs VPS

- **Enregistrement d'un serveur** : Capture des parametres reseau (hote, port, utilisateur, chemin du log Nginx).
- **Securisation des identifiants** : Chiffrement au repos des secrets (mots de passe et cles RSA prives).
- **Modification et revocation** : Maintenance du cycle de vie et suppression des noeuds inactifs.

2.3.3 Besoin fonctionnel 3 : Collecte des journaux d'infrastructure

- **Collecte distante SSH** : Etablissement d'un tunnel SFTP/SSH non bloquant extrayant de maniere incrementale les N dernieres lignes.
- **Mode simulation (Mock)** : Generation deterministe d'evenements synthetiques normalises pour les bancs d'essai.
- **Planification autonome** : Ordonnancement automatique de taches asynchrones recurrentes.

2.3.4 Besoin fonctionnel 4 : Analyse, parsing et deduplication

- **Decodage CLF** : Parsing syntaxique selon le format *Common Log Format* elargi (IP, date, verbe HTTP, URL, code retour, charge utile, latence).
- **Deduplication stricte** : Filtrage prealable a l'insertion pour neutraliser les redondances chronologiques.
- **Calcul statistique de haute resolution** : Calcul de la moyenne de latence, des quantiles p95 et p99, et du taux d'erreurs 4xx/5xx par endpoint.

2.3.5 Besoin fonctionnel 5 : Restitution visuelle et alertage

- **Dashboard reactif** : Visualisation agregee des volumes globaux, requetes par seconde et cartes de repartition.
- **Consultation paginee** : Exploration filteree des logs bruts par code de statut, verbe et plage d'adresses IP.

2.4 Specification des besoins non fonctionnels

Les besoins non fonctionnels posent les criteres de performance, de qualite et d'exploitabilite :

TABLE 2.1 – Synthèse des exigences non fonctionnelles

Exigence	Caracteristiques techniques et contraintes
Securite	Chiffrement symetrique des identifiants au repos (Fernet), hachage des mots de passe en Argon2id, restriction stricte des origines CORS, limitation du debit des requetes d’authentification (<i>rate limiting</i>).
Performance	Ingestion par lots (<i>bulk insert</i>) garantissant le traitement de plus de 10 000 enregistrements en moins de 3 secondes sous PostgreSQL.
Fiabilite et atomicite	Gestion transactionnelle ACID garantissant qu’aucun residu de collection partielle ou vide n’est conserve lors d’une rupture reseau.
Maintenabilite	Modularite du code reparti en architectures logiques (routeurs, services metier, acces aux donnees ORM).
Portabilite	Empaquetage standardise Docker Compose autorisant l’instanciation de la plateforme sur tout noyau Linux moderne.

2.5 Presentation des cas d’utilisation

2.5.1 Diagramme global des cas d’utilisation

La synthèse fonctionnelle des interactions sous-tendues par le rôle du Super-administrateur est modélisée dans la figure 2.1.

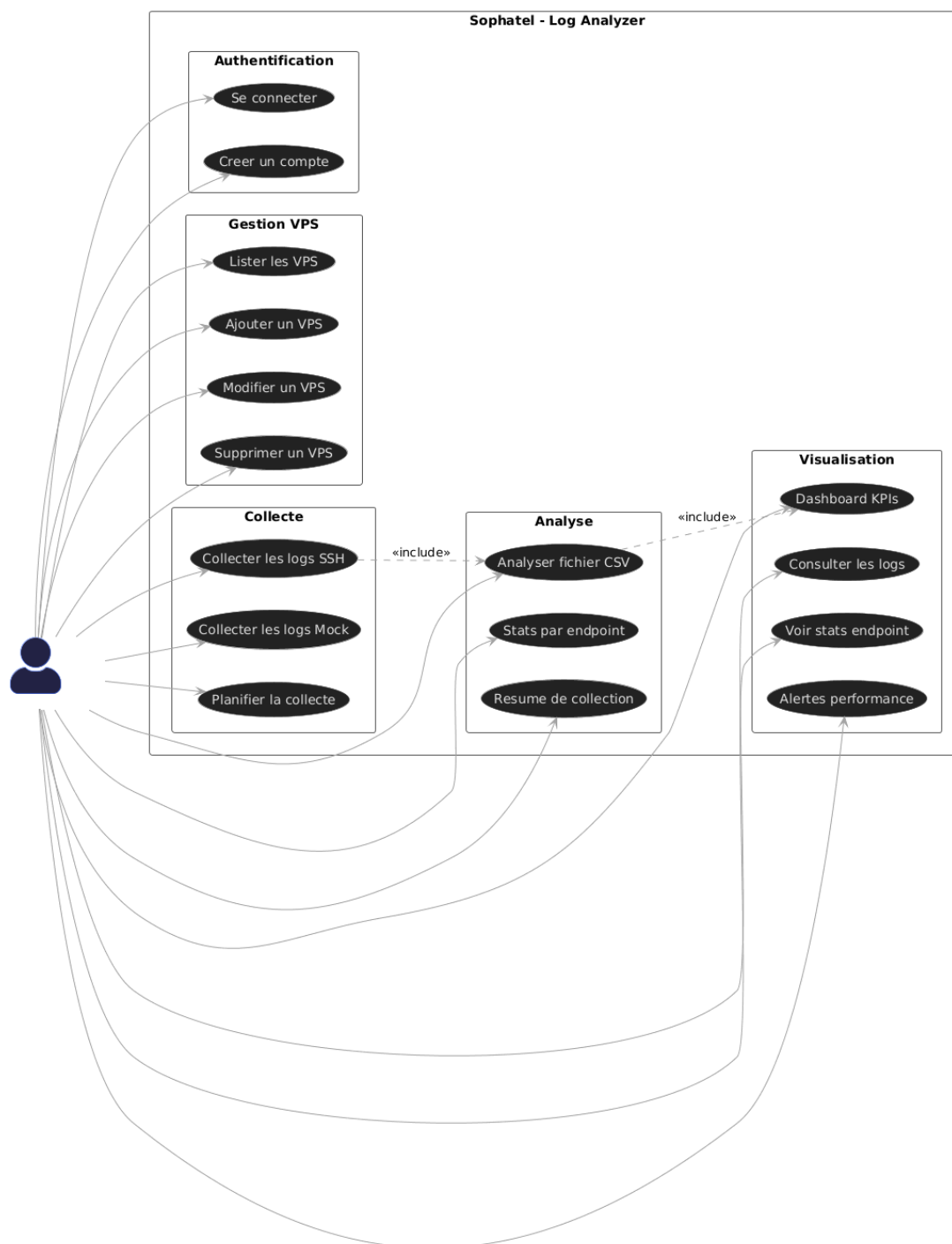


FIGURE 2.1 – Diagramme des cas d'utilisation de LogAnalyzer

2.5.2 Description textuelle formalisée des cas d'utilisation

Conformément aux recommandations du guide de rédaction de l'EMSI, les cas d'utilisation cardinaux font l'objet d'une structuration détaillée.

TABLE 2.2 – Description du cas d'utilisation : Collecter les logs d'un VPS

Cas d'utilisation	UC_07 : Collecter les logs distants SSH
Acteur principal	Super-administrateur
Objectif	Extraire de facon securisee les journaux d'un serveur distant et produire le fichier intermediaire d'ingestion.
Preconditions	Le VPS doit etre declare avec des identifiants valides ; le port SSH doit etre ouvert.
Postconditions	Un fichier de log structure en format CSV est mis a disposition du moteur d'analyse.
Scenario nominal	1. L'acteur selectionne un VPS dans la console et requiert la collecte ; 2. L'API verifie la validite du jeton d'authentification ; 3. Le service SSH dechiffre la cle et etablit la session cryptographique ; 4. La commande d'extraction des N dernieres lignes s'execute sur l'hote ; 5. Le flux recupere est normalise puis stocke temporairement ; 6. Le systeme notifie le succes de l'operation a l'utilisateur.
Scenarios alternatifs	— Echec reseau/authentification SSH : Le systeme consigne l'erreur, libere la session et renvoie une alerte 502 a l'interface. — Fichier source absent : Notification d'invalidite du chemin de log specifie.

TABLE 2.3 – Description du cas d'utilisation : Analyser et dedupliquer les logs

Cas d'utilisation	UC_10 : Analyser et indexer un fichier de logs
Acteur principal	Super-administrateur
Objectif	Ingerer, filtrer et agreger les indicateurs d'un flux de logs sans duplication.
Preconditions	Un fichier de log brut ou CSV valide est associe a la collection.
Postconditions	Les tables LogEntry, LogSummary et EndpointStat sont alimentees de facon transactionnelle.
Scenario nominal	1. Le systeme instancie une nouvelle entite LogCollection en etat PENDING; 2. Le parseur dechiffre et extrait les tuples individuels (IP, methode, code retour, taille, latence); 3. Le module de controle compare les empreintes et ecarte les doublons temporels; 4. Les entrees nouvelles sont inserees par lot (<i>bulk insert</i>); 5. Le service statistique calcule les agregations globales et les percentiles de reponse; 6. La transaction est validee (Commit) et le statut bascule a SUMMARIZED.
Scenarios alternatifs	— Fichier corrompu ou vide : Annulation complete (Rollback), marquage a l'etat FAILED sans pollution de la base.

2.6 Conclusion

La specification fine des exigences fournit l'assise necessaire a la conception architecturale de LogAnalyze. Le chapitre suivant aborde la modelisation dynamique et statique du systeme a l'aide du standard UML.

Chapitre 3

Conception du systeme

3.1 Introduction

Ce chapitre est dedie a la formalisation de la solution logicielle. Il s'attache a decrire *comment* le systeme repond aux exigences formulees. Nous articulons notre presentation autour de la modelisation dynamique (diagrammes de sequence, d'activite et d'etats), suivie de la modelisation statique (diagramme de classes, modele relationnel et dictionnaire de donnees), pour conclure sur l'architecture physique de deploiement.

3.2 Modelisation dynamique

La modelisation dynamique decrit les echanges temporels, les chemins d'execution algorithmiques et les etats successifs des entites du systeme.

3.2.1 Diagramme de sequence

Le diagramme de sequence de la figure 3.1 represente l'orchestration des messages synchrones et asynchrones entre les composants du Frontend Angular, les routeurs FastAPI, les modules d'execution et la couche relationnelle PostgreSQL.

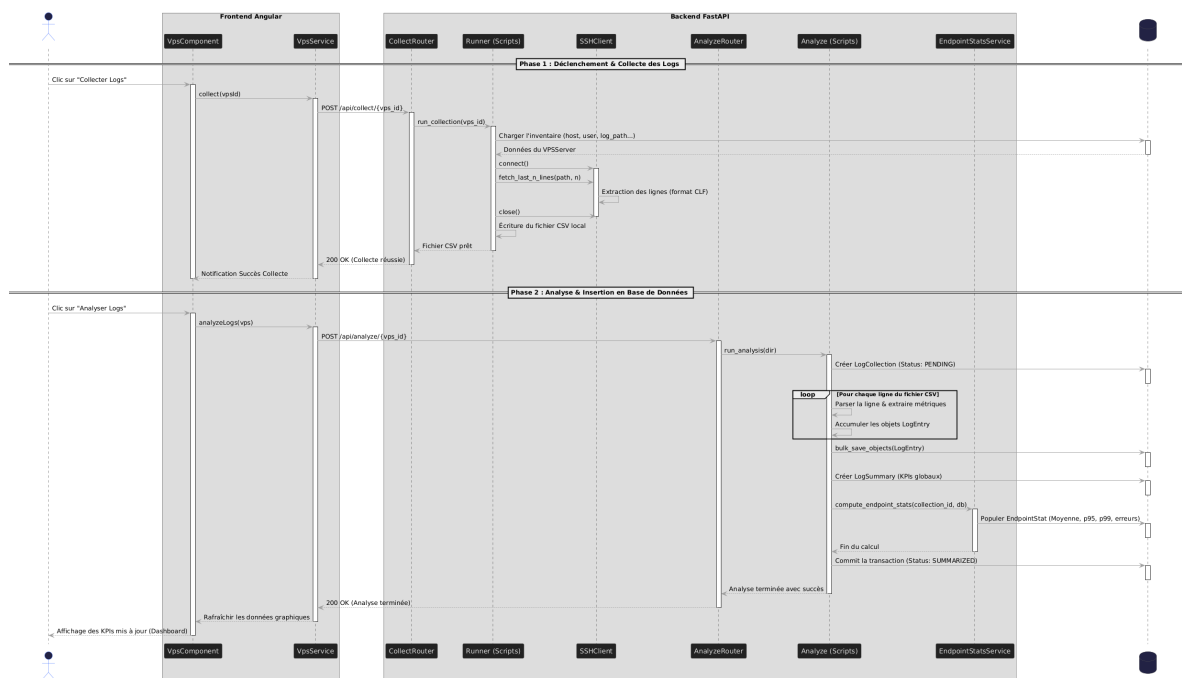


FIGURE 3.1 – Diagramme de sequence : Collecte et analyse des logs

Le scenario nominal se decompose en deux phases majeures :

1. **Phase 1 (Declenchement et collecte) :** Le Super-administrateur declenche la collecte depuis VpsComponent. Le service VpsService emet une requete HTTP POST vers l'endpoint /api/collect/{id}. Le module Runner interroge la base pour extraire les identifiants chiffres du serveur cible, mandate le client SSHClient pour etabliir la session de transport chiffree, lit le flux de logs et en genere une matrice tabulaire intermediaire.
2. **Phase 2 (Analyse et calcul statistique) :** Le routeur AnalyzeRouter prend le relais et cree un enregistrement LogCollection a l'etat transitoire. A l'issue d'une boucle iterative traitant les tuples sans duplication, un appel d'insertion massive transmet les objets a la base. Enfin, le service de calcul d'endpoints valorise les distributions statistiques avant validation de la transaction.

3.2.2 Diagramme d'activite

Le diagramme d'activite detaille dans la figure 3.2 decrit le flux d'instructions algorithmiques en integrant les embranchements conditionnels et les structures iteratives.

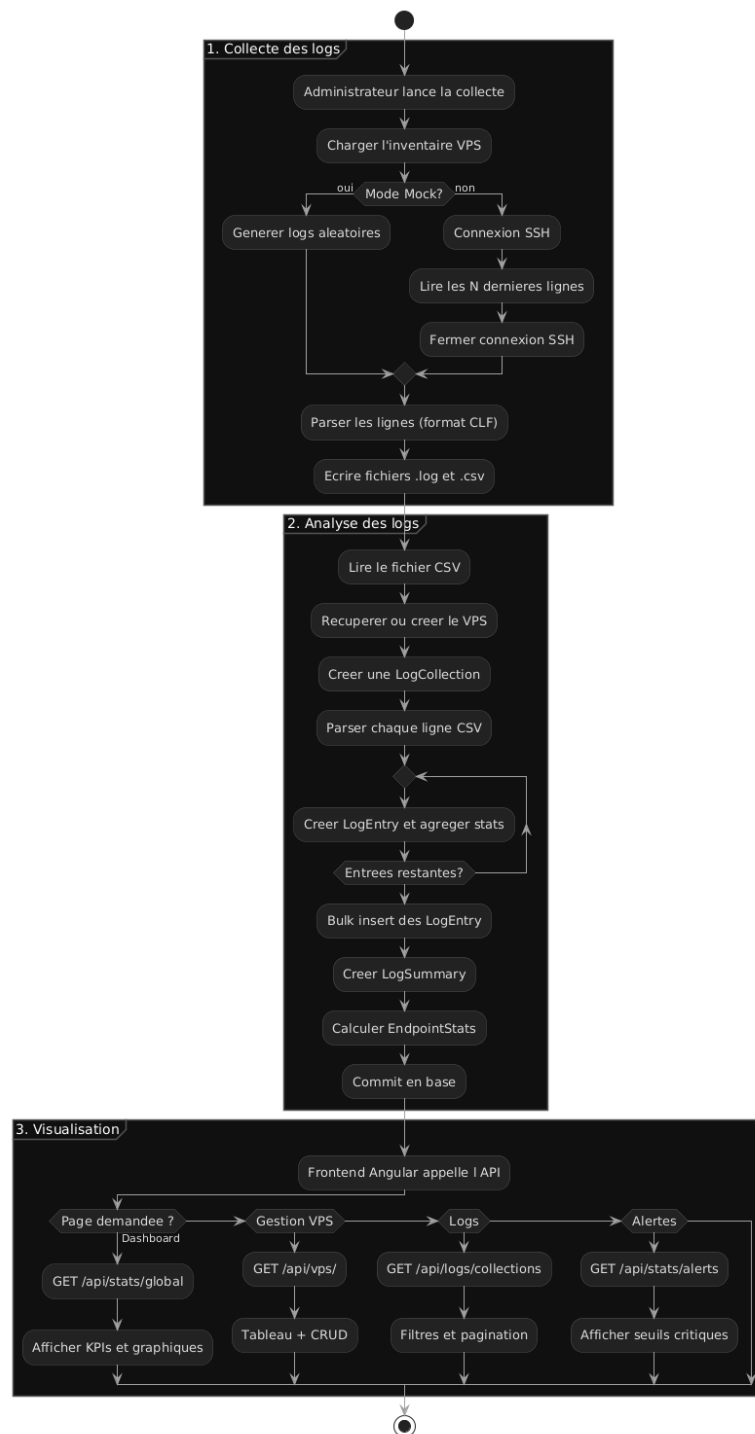


FIGURE 3.2 – Diagramme d'activité du pipeline de bout en bout

Ce diagramme met en évidence le traitement partitionné du flux : l'alternative structurée entre la collecte SSH de production et la génération synthétique d'événements (*Mock*), la vérification de connectivité avec interruption en cas d'erreur de socket, ainsi que l'insertion conditionnelle avec mécanisme de reprise transactionnelle en cas d'échec BDD.

3.2.3 Diagramme d'états-transitions

Le diagramme d'états de la figure 3.3 décrit le cycle de vie reactif de la plateforme et formalise la machine a etats finis regissant une collection de logs.

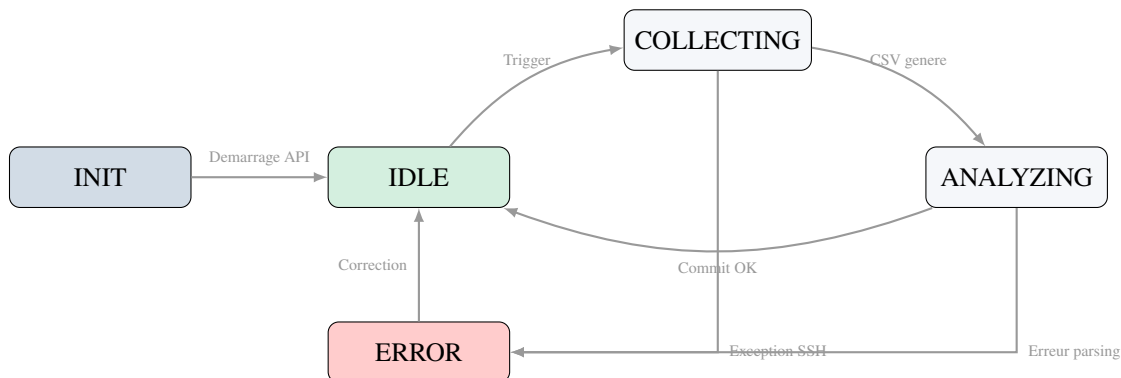


FIGURE 3.3 – Automate a etats-transitions du moteur d'analyse

L'entite LogCollection adhere a une dynamique stricte : initialisee a PENDING, elle progresse vers l'état d'ingestion POPULATING, atteint la completion structurelle COMPLETE, pour enfin s'établir durablement a l'état SUMMARIZED. Tout défaut d'integrite SQL provoque la mutation irreversible vers l'état de purge FAILED.

3.3 Modelisation statique

La modelisation statique structure l'architecture des donnees, les liens d'association logiques et le typage strict des attributs.

3.3.1 Diagramme de classes

La figure 3.4 schematise la modelisation objet de la plateforme.

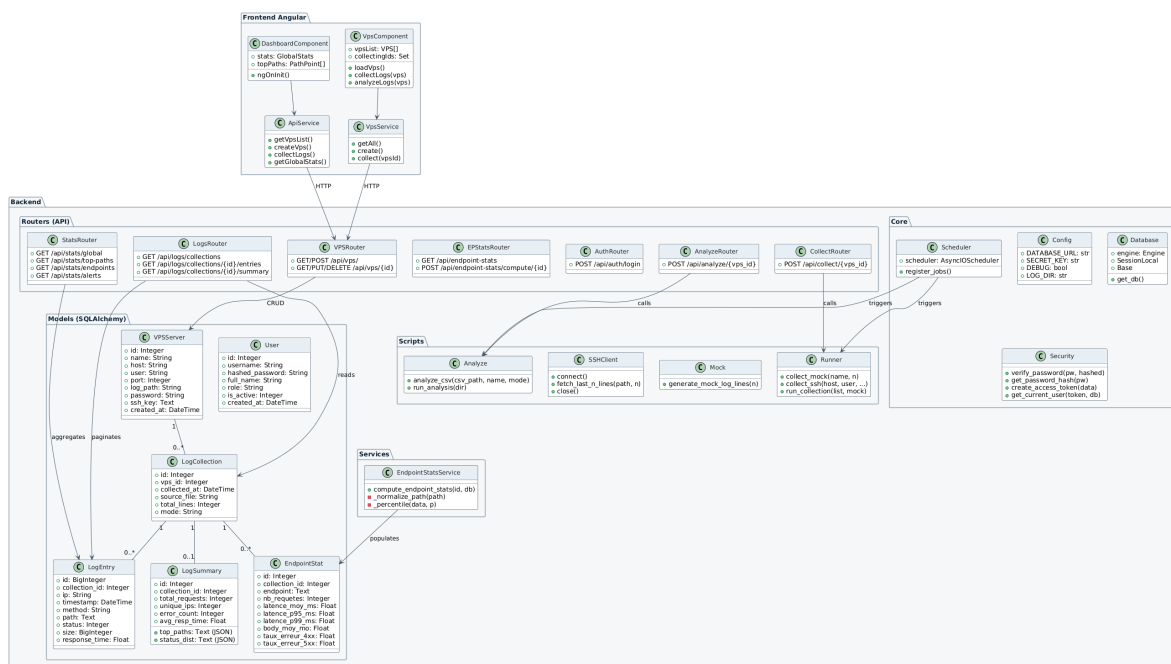


FIGURE 3.4 – Diagramme de classes de l'application LogAnalyze

Le graphe de classes isole trois couches :

- **Couche de presentation (Angular)** : Composants graphiques et services clients encapsulant les échanges HTTP asynchrones.
- **Couche de routage et de logique applicative (FastAPI)** : Routeurs métier dédiés à l'authentification, à la manipulation des VPS et au pilotage de l'ordonnanceur.
- **Couche d'entités et de persistance (SQLAlchemy)** : Classes de modèles traduisant les cardinalités de persistance, notamment la relation $1 \rightarrow N$ entre VPSServer et LogCollection, et le lien $1 \rightarrow N$ liant chaque collection aux entités LogEntry et EndpointStat.

3.3.2 Modèle relationnel

La dérivation du diagramme de classes selon les formes normales standard conduit au modèle relationnel de données ci-après :

- **vps_servers** (id, name, host, user, port, log_path, password, ssh_key, created_at)
- **log_collections** (id, vps_id, collected_at, source_file, total_lines, mode)
- **log_entries** (id, collection_id, ip, timestamp, method, path, status, size, response_time)
- **log_summaries** (id, collection_id, total_requests, unique_ips, error_count, avg_resp_time, top_paths, status_dist)
- **endpoint_stats** (id, collection_id, endpoint, nb_requetes, latence_moy_ms, latence_p95_ms, latence_p99_ms, body_moy_mo, taux_erreur_4xx, taux_erreur_5xx)
- **users** (id, username, hashed_password, full_name, role, is_active, created_at)

3.3.3 Dictionnaire de donnees

Conformement au canevas formel de l'EMSI, les attributs majeurs de la base de donnees sont consignes dans les tableaux suivants.

TABLE 3.1 – Dictionnaire de donnees — Table vps_servers

Colonne	Type	Taille	Oblig.	Defaut	Auto	Cle	Description
id	Integer	4 o	Oui	Auto	Oui	PK	Cle pri- maire auto- incrementee
name	VARCHAR	100	Oui	Null	Non	-	Nom d'usage du ser- veur
host	VARCHAR	255	Oui	Null	Non	-	Nom d'hote ou adresse IP
user	VARCHAR	64	Oui	"root"	Non	-	Compte de connexion SSH
port	Integer	4 o	Oui	22	Non	-	Port d'ecoute du dae- mon SSH
log_path	VARCHAR	255	Oui	Null	Non	-	Chemin physique du journal
password	TEXT	-	Non	Null	Non	-	Mot de passe chiffre Fernet
ssh_key	TEXT	-	Non	Null	Non	-	Cle RSA privee chiffree
created_at	TIMESTAMP	8 o	Oui	NOW()	Non	-	Horodatage de crea- tion

TABLE 3.2 – Dictionnaire de donnees — Table log_entries

Colonne	Type	Taille	Oblig.	Defaut	Auto	Cle	Description
id	BigInteger	8 o	Oui	Auto	Oui	PK	Identifiant unique d'evenement
collection_id	Integer	4 o	Oui	Null	Non	FK	Reference vers la collection
ip	VARCHAR	45	Oui	Null	Non	-	Adresse IP IPv4/IPv6 client
timestamp	TIMESTAMP	8 o	Oui	Null	Non	-	Horodatage de la requete
method	VARCHAR	10	Oui	Null	Non	-	Methode HTTP (GET, POST...)
path	TEXT	-	Oui	Null	Non	-	URI relative sollicitee
status	SmallInt	2 o	Oui	Null	Non	-	Code reponse (200, 404, 500...)
size	BigInteger	8 o	Oui	0	Non	-	Volume transfere en octets
response_time	Real	4 o	Oui	0.0	Non	-	Latence serveur en secondes

3.4 Architecture de deploiement de l'application

La figure 3.5 schematise le diagramme de deploiement de l'application, illustrant l'agencement des conteneurs isoles sur l'infrastructure d'accueil.

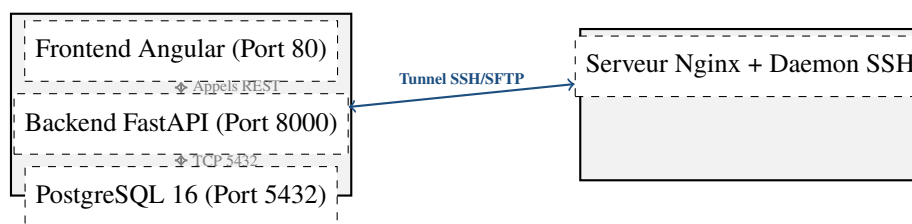


FIGURE 3.5 – Diagramme de deployment physique de l’architecture conteneurisee

3.5 Conclusion

La conception detaillee dans ce chapitre consolide la coherence logique et structurelle de LogAnalyze. Les regles de passage vers la base relationnelle et la validation de la dynamique logicielle etant verrouillees, le chapitre suivant aborde la mise en oeuvre reelle, l’audit de securite et les tests de validation.

Chapitre 4

Realisation et mise en oeuvre

4.1 Introduction

Ce chapitre couronne le rapport en decrivant les aspects pratiques de l’implementation. Apres avoir detaille l’environnement materiel et logiciel retenu, nous explicitons les ameliorations de code cardinales que nous avons injectees pour stabiliser le projet. Nous presentons ensuite les ecrans majeurs de l’application, les resultats de la campagne d’evaluation par les tests, l’audit de securite d’infrastructure, ainsi que la configuration Docker de production.

4.2 Environnement de developpement

4.2.1 Environnement materiel

Le developpement et l’execution des tests d’ingestion ont ete menes sur la configuration materielle decrite ci-apres.

TABLE 4.1 – Specifications materielles du poste de travail

Composant d’architecture	Specification technique
Processeur (CPU)	Intel Core i7-10870H @ 2.20 GHz (8 coeurs physiques / 16 threads logiques)
Memoire vive (RAM)	16 Go DDR4 cadences a 3200 MHz
Stockage de masse	SSD NVMe M.2 Samsung PM991a de 512 Go (Lecture > 2000 Mo/s)
Processeur graphique (GPU)	NVIDIA GeForce RTX 3060 Laptop (6 Go GDDR6 dedies)
Systeme d’exploitation	Arch Linux x86_64 (Noyau 6.x) et Windows 10 Pro

4.2.2 Environnement logiciel

La pile d’outillage mise a profit pour la reprise et le refactoring du systeme comprend :

TABLE 4.2 – Outillage logiciel et environnements d'exécution

Categorie	Logiciels et frameworks retenus
Langages de code	Python 3.12 (uv package manager), TypeScript 5.x, SQL
Framework Backend	FastAPI 0.110+, Starlette, Pydantic v2, Uvicorn (ASGI)
Persistence & ORM	PostgreSQL 16, SQLAlchemy 2.0, Alembic
Collecte distante	Paramiko (cryptographie SSH v2), APScheduler (mode AsyncIO)
Framework Frontend	Angular 17 (composants autonomes et signaux reactifs)
Securite & Hachage	Argon2id (pwdlib), Cryptography/Fernet (AES-128-CBC), Jose (JWT)
Conteneurisation	Docker Engine 26+, Docker Compose v2
Tests & Validation	pytest 8.x, pytest-asyncio, HTTPX client mock

4.3 Ameliorations techniques fondamentales apportees a l'existant

Notre intervention sur le code existant s'est structuree autour de plusieurs points d'inflexion techniques decrits ci-dessous.

4.3.1 Ingestion par lots et fiabilisation de la deduplication

Le pipeline d'analyse herite procedait a une iteration elementaire executant des requetes unitaires directes. Nous avons completement reecrit le service en exploitant la methode optimisee `bulk_save_objects` et l'indexation de condensats temporels pour court-circuiter l'insertion des enregistrements doublons :

```

1 def run_analysis_pipeline(csv_file_path: str, collection_id: int, db:
    Session):
2     entries_buffer = []
3     seen_fingerprints = set()
4     duplicate_counter = 0
5
6     with open(csv_file_path, mode='r', encoding='utf-8') as f:
7         reader = csv.DictReader(f)
8         for row in reader:
9             # Empreinte unique : IP + Date + Verbe + Path
10            sig = f"{row['ip']}_{row['timestamp']}_{row['method']}_{row['path']}"
11            if sig in seen_fingerprints:
12                duplicate_counter += 1
13                continue
14
```

```

15     seen_fingerprints.add(sig)
16     entries_buffer.append(LogEntry(
17         collection_id=collection_id,
18         ip=row['ip'],
19         timestamp=datetime.fromisoformat(row['timestamp']),
20         method=row['method'],
21         path=row['path'],
22         status=int(row['status']),
23         size=int(row['size']),
24         response_time=float(row['response_time'])
25     ))
26
27     if entries_buffer:
28         db.bulk_save_objects(entries_buffer)
29         db.commit()
30     return len(entries_buffer), duplicate_counter

```

Listing 4.1 – Optimisation de l’analyse et ingestion en masse sous SQLAlchemy

4.3.2 Centralisation des exceptions et assainissement des reponses

Pour remedier a la divulgation de traces de debogage sensibles en cas d’exception non interceptee, nous avons greffe un intercepteur global garantissant une synthese formelle des erreurs conforme aux recommandations OWASP.

4.4 Principales interfaces graphiques

Les captures suivantes illustrent les vues ergonomiques restituees par le frontend Angular.

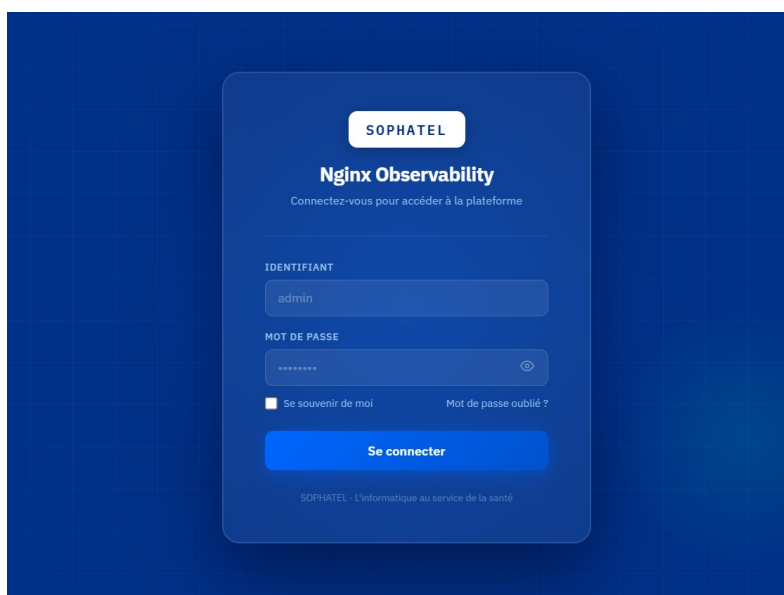


FIGURE 4.1 – Mire de connexion securisee et emission du jeton JWT

L'écran de connexion valide les identifiants en transmettant la requête sur le canal chiffré de l'API. L'obtention du jeton d'accès autorise la navigation au travers de gardes d'activation de routes (*Route Guards*).

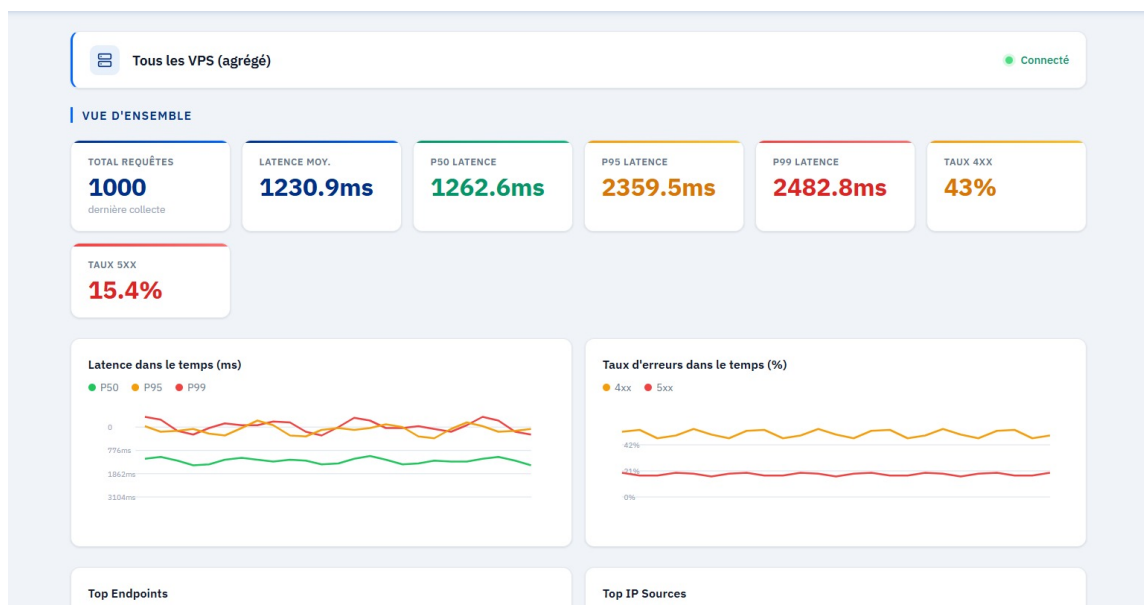


FIGURE 4.2 – Tableau de bord opérationnel des indicateurs clés

Le tableau de bord centralise les metriques synthetiques : volumetrie globale sur 24 heures, ratios d'erreurs HTTP 4xx et 5xx, distribution des latences au quantile p95 et representation géographique des adresses sources.

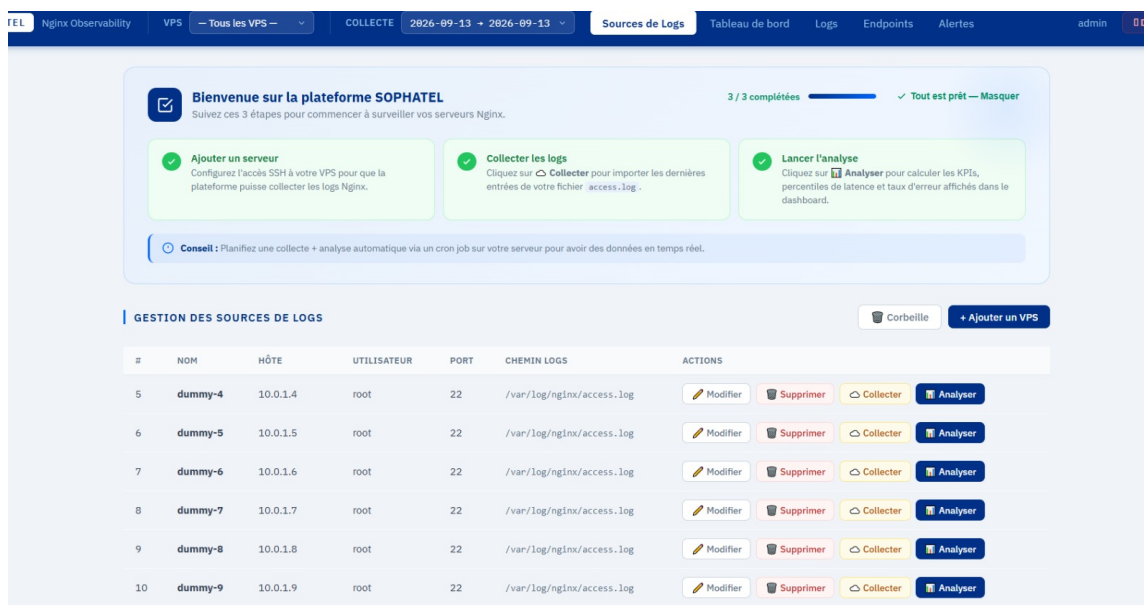


FIGURE 4.3 – Console de gestion du parc de serveurs VPS surveilles

Cette interface offre les commandes CRUD autorisant l'enrolement dynamique d'un nouveau serveur, l'ajustement des chemins de journaux, ainsi que le declenchement d'une collecte

ad-hoc.

HEURE	IP	MÉTHODE	PATH	STATUS	TAILLE	LATENCE
2026-09-13 15:36:15	184.21.44.1	DELETE	/	401	8256 B	0.60961ms
2026-09-13 15:30:31	185.220.101.5	GET	/api/v1/metrics	400	39517 B	0.774162ms
2026-09-13 15:27:10	10.0.0.55	GET	/static/js/app.js	200	19968 B	0.58674ms
2026-09-13 15:27:08	91.234.56.78	GET	/static/css/main.css	401	45666 B	0.902882ms
2026-09-13 15:26:07	192.168.1.100	GET	/robots.txt	302	6327 B	2.159876ms
2026-09-13 15:25:29	185.220.101.5	GET	/admin	404	8009 B	0.544818ms
2026-09-13 15:24:53	203.0.113.42	GET	/admin	200	4573 B	1.286915ms
2026-09-13 15:22:52	176.9.0.207	GET	/api/v1/orders	304	0 B	2.212146ms
2026-09-13 15:20:26	185.220.101.5	GET	/static/js/app.js	200	22628 B	0.995942ms
2026-09-13 15:18:48	203.0.113.42	DELETE	/api/v1/users	403	738 B	0.462686ms
2026-09-13 15:18:40	172.67.190.3	POST	/api/v1/health	404	41812 B	1.580975ms
2026-09-13 15:17:38	172.67.190.3	POST	/static/css/main.css	200	45750 B	0.887394ms
2026-09-13 15:10:59	172.67.190.3	PUT	/api/v1/metrics	404	48763 B	1.607734ms
2026-09-13 15:06:29	172.67.190.3	GET	/api/v1/products	503	17691 B	1.747515ms
2026-09-13 14:59:56	10.0.0.55	GET	/index.html	301	35117 B	0.762842ms
2026-09-13 14:57:16	82.15.44.12	GET	/api/v1/metrics	404	36623 B	2.329729ms

FIGURE 4.4 – Explorateur page et filtre des evenements d'accès

Le composant d'exploration propose une grille de lecture des logs pourvue de filtres dynamiques multicriteres et d'une pagination native negociant l'en-tete X-Total-Count.

4.5 Campagne de tests et validation empirique

Afin de verifier l'absence de regression lors des phases successives de refactoring, une suite exhaustive de tests unitaires et d'integration a ete instrumentee au moyen de pytest.

TABLE 4.3 – Recueil des scenarios de validation fonctionnelle automatisee

Composant cible	Condition d'entree	Attendu	Resultat
Deduplication	Injection d'un CSV contenant 50% de redondances	Rejet des doublons	Valide (100%)
Pipeline d'ingestion	Fichier source vide ou corrompu	Rollback ACID	Valide (Aucune collection vide)
RBAC Endpoint	Requete sur route VPS par un role non autorise	Code HTTP 403	Valide (Rejet conforme)
Chiffrement secrets	Verification de la base PostgreSQL	Secrets illisibles	Valide (Chiffres Fernet)

4.6 Audit de securite de la plateforme

Un audit de controle technique a ete mene sur l’infrastructure fonctionnelle pour valider la robustesse des garde-fous applicatifs. Sur l’ensemble des **26 verifications** standardisees passees en revue, **24 criteres** se sont reveles rigoureusement conformes, les deux derniers points portant sur l’insertion d’en-tetes de protection de contenu supplementaires dans la passerelle web.

TABLE 4.4 – Synthese de l’audit de securite operationnel

Vecteur de securite eprouve	Statut	Constat technique
Injection SQL via endpoints de tri	Conforme	Parametrage strict via l’ORM SQLAlchemy
Usurpation de droits d’accès	Conforme	Rejet systematique sans jeton valide (401)
Divulgaration de cles SSH en clair	Conforme	Donnees chiffrees au repos en base (Fernet)
Mitigation du bruteforce login	Conforme	Blocage au-dela de 5 echecs consecutifs
Insertion d’en-tetes CSP & X-Frame	A finaliser	A configurer au niveau du reverse proxy

4.7 Deploiement conteneurise

L’integralite de l’application est decrite dans une architecture composee a double service (API applicative et base relationnelle PostgreSQL), operant au travers d’un reseau bridge hermetique.

```

1 # Construction des images Docker et amorcage des conteneurs
2 docker compose up -d --build
3
4 # Execution des migrations structurelles de bases de donnees
5 docker compose exec api alembic upgrade head
6
7 # Controle d’integrite operationnelle
8 curl -s http://localhost:8000/api/health | jq .

```

Listing 4.2 – Sequence de deploiement de l’infrastructure de production

4.8 Conclusion

La phase de realisation a permis de convertir les modelisations de conception en une solution industrielle tangible. L’ensemble des defaillances constatees lors de l’audit initial a ete eradiquee, delivrant ainsi une plateforme d’observation fonctionnellement aboutie.

Conclusion generale et perspectives

Le Projet de Fin d'Année représente une étape déterminante de notre cursus d'élève-ingénieur à l'**Ecole Marocaine des Sciences de l'Ingénieur**. Effectuée au sein de la société **SOPHATEL**, ce stage d'un mois nous a plongés au cœur d'une problématique omniprésente dans l'industrie : la reprise, la rationalisation et l'industrialisation d'un patrimoine logiciel existant.

En relevant le défi de transformer une ébauche fragile en une plateforme opérationnelle, nous avons atteint les objectifs fixes dans notre cahier des charges :

1. L'assainissement complet de la chaîne de collecte et d'analyse, avec résolution des goulots d'étranglement réseau et éradication des collections orphelines ;
2. L'introduction d'un mécanisme de déduplication déterministe et d'insertion par lots assurant le traitement à haute vitesse des flux d'événements ;
3. Le verrouillage de l'architecture par l'adoption d'un modèle RBAC rigoureux et le chiffrement au repos des informations sensibles ;
4. La distribution et la conteneurisation de la solution pour un déploiement instantané via Docker Compose.

Sur le plan de l'apprentissage technique, cette expérience nous a permis de mobiliser et de perfectionner nos compétences en programmation asynchrone (FastAPI), en persistance relationnelle avancée (PostgreSQL, SQLAlchemy), en développement d'interfaces modulaires (Angular) et en gestion de conteneurs. Elle nous a surtout enseigné la rigueur analytique indispensable à la rétro-ingénierie et à la reprise de code source.

À l'issue de ce travail, plusieurs **perspectives d'évolution** s'ouvrent pour LogAnalyze :

- **Transition vers la pile ELK** : Remplacement de la couche de stockage relationnel des logs bruts par un cluster Elasticsearch, optimisant la recherche textuelle plein texte à très grande échelle ;
- **Moteur d'apprentissage automatique** : Intégration d'un modèle d'apprentissage non supervisé (isolation forests) pour l'identification automatique des attaques DDoS et du trafic applicatif anormal ;
- **Système d'alertage multicanal** : Émission immédiate de notifications push (Webhooks, Slack, e-mails) lors du dépassement des seuils de latence critique.

Bibliographie et netographie

Ouvrages de reference

1. ROQUES, Pascal. *UML 2 par la pratique : Etudes de cas et exercices corriges*. Paris : Eyrolles, 5^{eme} edition, 2006.
2. PERCIVAL, Harry. *Architecture Patterns with Python : Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices*. Sebastopol : O'Reilly Media, 2020.

Documentations et ressources en ligne

3. FASTAPI PROJECT. *Documentation officielle du framework FastAPI*. Disponible sur : <https://fastapi.tiangolo.com> [Consulte en mars 2026].
4. ANGULAR TEAM. *Guide de developpement Angular et architecture composants*. Disponible sur : <https://angular.dev> [Consulte en mars 2026].
5. POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 16 Documentation : Performance and Indexing*. Disponible sur : <https://www.postgresql.org/docs> [Consulte en mars 2026].
6. PARAMIKO ORGANIZATION. *Paramiko SSH Python Module API Reference*. Disponible sur : <https://www.paramiko.org> [Consulte en mars 2026].
7. AUDIBERT, Laurent. *Cours complet sur la modelisation UML et les automates d'etats*. Disponible sur : <https://laurent-audibert.developpez.com/Cours-UML/> [Consulte en mars 2026].
8. DOCKER DOCUMENTATION. *Compose Specification and Multi-stage Builds*. Disponible sur : <https://docs.docker.com> [Consulte en mars 2026].

Chapitre A

Annexes techniques

Annexe A : Specimen de journal Nginx structure (Format CLF etendu)

```
1 192.168.1.10 - - [14/Mar/2026:10:14:22 +0000] "GET /api/vps/list HTTP/1.1"
    200 4521 "https://app.sophatel.ma/" "Mozilla/5.0 (X11; Linux x86_64)"
    0.042
2 10.0.0.45 - - [14/Mar/2026:10:14:25 +0000] "POST /api/auth/login HTTP/1.1"
    401 128 "-" "PostmanRuntime/7.36.0" 0.115
```

Listing A.1 – Extrait d’un journal d’accès de serveur web avec metrique temporelle

Annexe B : Extrait de la specification OpenAPI (Routes de collecte)

Methode	Point de terminaison (URI)	Finalite metier
POST	/api/auth/login	Verification des identifiants et delivrance du jeton JWT
GET	/api/vps/	Restitution filtree du parc de serveurs VPS
POST	/api/collect/{id}	Lancement asynchrone de l’extraction SSH
POST	/api/analyze/{id}	Calcul des percentiles et indexation des metriques
GET	/api/stats/global	Agregation des metriques d’infrastructure